MailServer

Self-hosted Email, Calendar, Contacts & Files Platform

Document Type	Date	Stack	Platform
Developer Handover & Operations Guide	**April 2026**	**React · Node.js · Docker · MongoDB**	**Windows 11 / Raspberry Pi 4**

TABLE OF CONTENTS

1. Project Overview
2. Architecture Diagram
3. Directory Structure
4. Prerequisites & Setup
5. Running in Development
6. Running in Production (Docker)
7. Environment Variables
8. Frontend Guide
9. Backend Guide
10. Database Guide
11. Docker Reference
12. Making & Deploying Changes
13. User Roles & Login
14. File Storage
15. Office Editing (WOPI)
16. Common Errors & Fixes
17. Key Files Quick Reference
18. Checklist for New Developer

1. Project Overview

MailServer is a **fully self-hosted productivity suite** — a drop-in alternative to Gmail / G Suite that runs entirely on your own hardware. It provides webmail, shared calendar, contacts, file storage, and in-browser office document editing.

Webmail

Gmail-style 3-pane inbox. Compose, reply, folders, search. Powered by Dovecot (IMAP) + Postfix (SMTP).

Calendar

CalDAV sync. Shared team events. FullCalendar UI with drag-and-drop.

Contacts

CardDAV-based contact management. Syncs with phone / Outlook / Thunderbird.

Files

Upload, browse, share files & folders (Google Drive style). Stored in `D:\maildata\files\`.

Office Editing

Edit .docx / .xlsx / .pptx in-browser via Collabora CODE (WOPI protocol). No desktop app needed.

Multi-tenant

Superadmin creates company tenants. Each company admin manages their own users.

2. Architecture Diagram

```
  Browser
     │  port 80 (Docker) or 5173 (dev)
     ▼
┌──────────────────────────────────────────────────────────┐
│  Nginx  (reverse proxy)  — port 8080/8443                 │
│  Routes:  /api/*  →  api:3000                             │
│           /*      →  webmail:80 (React SPA)               │
└──────────────────┬───────────────────────────────────────┘
                   │ Docker network: mailnet
     ┌─────────────┴──────────────┐
     ▼                            ▼
┌──────────┐            ┌──────────────┐
│  webmail │            │  api (Node.js) │
│  :80     │            │  :3000         │
│  (nginx  │            │  Express +     │
│   serve  │            │  JWT auth      │
│   React  │            └──────┬─────────┘
│   build) │                   │
└──────────┘          ┌────────┴─────────┐
                      ▼        ▼         ▼
               ┌──────────┐ ┌──────┐ ┌──────────┐
               │ MongoDB  │ │Redis │ │Dovecot   │
               │ :27017   │ │:6379 │ │IMAP:993  │
               └──────────┘ └──────┘ └──────────┘

                                     ┌──────────┐
                                     │ Postfix  │
                                     │ SMTP:587 │
                                     └──────────┘

                                     ┌──────────┐
                                     │Collabora │
                                     │ :9980    │
                                     └──────────┘

  Files stored at: D:\maildata\files\  (Windows host)
  Mounted inside api container at: /data/files
```

> In **development**, you bypass Docker and run the backend on `localhost:3000` and Vite dev server on `localhost:5173`. In **production**, everything runs in Docker containers.

3. Directory Structure

```
d:\Language prop\Mail\   ← project root
  ├── backend/   Node.js Express REST API
  │   ├── src/
  │   │   ├── index.ts   Entry point, Express app setup
  │   │   ├── config/   index.ts — all env vars via Zod schema
  │   │   ├── routes/   auth, mail, files, calendar, contacts, wopi…
  │   │   ├── models/   Mongoose: User, Tenant, Share, Bill…
  │   │   ├── middleware/   auth.ts (JWT verify), errorHandler.ts
  │   │   └── services/   imap, smtp, localFile, nextcloud
  │   ├── .env   ⚠ dev overrides (never commit secrets)
  │   ├── Dockerfile
  │   └── package.json
  ├── frontend/   React 18 + Vite + Tailwind
  │   ├── src/
  │   │   ├── App.tsx   Route definitions
  │   │   ├── pages/   Login, Inbox, Calendar, Files, Contacts, Settings…
  │   │   ├── components/   Layout (Sidebar, Header, AppNavbar), Mail, ui
  │   │   ├── api/   client.ts (axios + token refresh), mailApi, authApi…
  │   │   ├── store/   Zustand: useAuthStore, useMailStore, useUiThemeStore
  │   │   └── lib/   themes.ts, utils.ts, mailFolders.ts
  │   ├── Dockerfile
  │   ├── nginx.conf   Serves React build inside container
  │   └── vite.config.ts
  ├── config/   nginx, dovecot, postfix, rspamd, mongodb configs
  ├── docker/   Custom Dockerfiles for Dovecot, Postfix
  ├── docker-compose.yml   Core services: Dovecot, Postfix, MongoDB, Redis…
  ├── docker-compose.apps.yml   App layer: api, webmail, nginx, cloudflared
  ├── dev.sh   Start dev servers (bash dev.sh)
  ├── .env   Root secrets used by docker-compose
  └── D:\maildata\files\   User files (separate drive, mounted into api container)
```

4. Prerequisites & Initial Setup

Required Software

TOOL	VERSION	PURPOSE
Node.js	v20 LTS	Run backend and frontend build tools
Docker Desktop	Latest	Run all services in containers
Git	Any	Version control
VS Code	Recommended	Development IDE (has Claude Code extension)

First-Time Setup

1. **Clone or access the project:**
   `cd "d:\Language prop\Mail"`
2. **Install backend dependencies:**
   ```
   cd backend && npm install && cd ..
   ```
3. **Install frontend dependencies:**
   ```
   cd frontend && npm install && cd ..
   ```
4. **Ensure `D:\maildata\files\` directory exists** — this is where all user files are stored.
5. **Check `.env` files** — root `.env` (Docker secrets) and `backend/.env` (dev overrides) must exist.
6. **Start the Docker network first:**
   ```
   docker compose -f docker-compose.yml up -d
   ```

5. Running in Development Mode

> Development mode runs the backend and frontend directly on your machine (no Docker for app code). The core services (MongoDB, Redis, Dovecot, Postfix) still run in Docker.

Quick Start

```
# From project root in Git Bash / WSL
bash dev.sh

# First time (installs npm packages too):
bash dev.sh --install
```

This starts:

- Backend API on `http://localhost:3000`
- Frontend Vite dev server on `http://localhost:5173`

> ⚠️ Always access the app on **port 5173** during development, NOT port 80. Port 80 is the Docker container serving the old production build.

Starting Services Individually

```
# Backend only
cd backend
npm run dev          # tsx watch --env-file=.env src/index.ts

# Frontend only
cd frontend
npm run dev          # vite --host 0.0.0.0
```

Dev Login URLs

ROLE	URL
All users (user / admin / superadmin)	`http://localhost:5173/login`
Landing page	`http://localhost:5173/`
API health check	`http://localhost:3000/health`

6. Running in Production (Docker)

Full Stack Start (First Time)

```
# Step 1: Start core mail services
docker compose -f docker-compose.yml up -d

# Step 2: Build and start app layer
docker compose -f docker-compose.apps.yml up -d --build
```

After Making Code Changes (Most Common)

```
# Rebuild only changed containers
docker compose -f docker-compose.apps.yml build webmail api

# Restart them
docker compose -f docker-compose.apps.yml up -d webmail api
```

> ⚠️ Every time you change frontend or backend source code, you MUST rebuild the relevant Docker image and restart the container. Changes to `frontend/src/` require rebuilding `webmail`. Changes to `backend/src/` require rebuilding `api`.

Check Status

```
docker compose -f docker-compose.apps.yml ps
docker logs mailserver-api --tail=50
docker logs mailserver-webmail --tail=20
```

Stop Everything

```
docker compose -f docker-compose.apps.yml down
docker compose -f docker-compose.yml down
```

Access in Production

WHAT	URL
Web app (via nginx)	`http://<server-ip>:8080`
API direct (local only)	`http://127.0.0.1:3001`
HTTPS (Cloudflare Tunnel)	Your configured domain

7. Environment Variables

Root `.env` — Docker Compose Secrets

VARIABLE	EXAMPLE	PURPOSE
`JWT_SECRET`	long-random-string-min-32chars	Access token signing
`JWT_REFRESH_SECRET`	another-long-random-string	Refresh token signing
`MONGO_URI`	mongodb://mailserver:pass@mongodb:27017/mailserver?authSource=mailserver	MongoDB connection
`REDIS_PASSWORD`	redis-secret	Redis auth
`MAIL_DOMAIN`	yourdomain.com	Email domain
`INTERNAL_AUTH_TOKEN`	secret-token	Dovecot → API auth
`CLOUDFLARE_TUNNEL_TOKEN`	eyJh...	Cloudflare public HTTPS tunnel

`backend/.env` — Dev Overrides

```
# Connect to local MongoDB (no auth in dev)
MONGO_URI=mongodb://localhost:27017/mailserver
JWT_SECRET=dev-secret-key-at-least-32-characters
JWT_REFRESH_SECRET=dev-refresh-secret-at-least-32-chars
MAIL_DOMAIN=yourdomain.com
INTERNAL_AUTH_TOKEN=dev-internal-token
# File storage in dev
FILE_STORAGE_DRIVER=local
LOCAL_FILES_ROOT=D:/maildata/files
# Collabora / WOPI in dev
COLLABORA_URL=http://localhost:8082
WOPI_HOST=http://host.docker.internal:3000
```

8. Frontend Guide

The frontend is a **React 18** single-page application built with **Vite** and styled with **Tailwind CSS**.

Key Libraries

LIBRARY	PURPOSE
React Router v6	Client-side routing
TanStack Query (React Query)	Server state management, caching, background refetch
Zustand	Client state: auth, mail selection, UI theme
Axios	HTTP client with JWT interceptor & token refresh
Tiptap	Rich text email editor
FullCalendar	Calendar UI
Lucide React	Icons
Tailwind CSS	Utility-first CSS

State Management (Zustand Stores)

STORE	FILE	CONTAINS
`useAuthStore`	`store/index.ts`	accessToken, refreshToken, email, role, avatar, displayName
`useMailStore`	`store/index.ts`	selectedFolder, searchQuery, composeOpen
`useUiThemeStore`	`store/index.ts`	appBg (background theme colour)

Route Structure (`App.tsx`)

```
// Public routes — no auth needed
/                    → LandingPage
/login               → Login (universal for all roles)
/admin/login         → Login (same component)
/superadmin/login    → Login (same component)

// Protected routes — wrapped in RequireAuth → Layout
/mail/:folder        → Inbox (IMAP)
/calendar            → Calendar (CalDAV)
/contacts            → Contacts (CardDAV)
/files               → Files (local storage)
/settings            → Settings

// Admin only
/admin/users         → User management
/admin/guide         → Setup guide

// Superadmin only
/superadmin/tenants  → Tenant management
/superadmin/billing  → Billing
```

Adding a New Page

1. Create `frontend/src/pages/MyPage.tsx`
2. Import it in `App.tsx`
3. Add a `<Route>` inside the protected `<Layout>` route
4. Add a nav link in `components/Layout/AppNavbar.tsx` and/or `Sidebar.tsx`

Tailwind & Dynamic Colors

> ⚠️ Tailwind JIT purges unused classes at build time. If you construct class names dynamically (e.g. `` `bg-${color}-500` ``), Tailwind won't include them. Always use **inline styles** for dynamic colors: `style={{ backgroundColor: color }}` where `color` is a hex string.

Adding a New API Call

1. Add function to the relevant `frontend/src/api/xxxApi.ts` file
2. Use with `useQuery` (read) or `useMutation` (write) from TanStack Query
3. The axios client in `api/client.ts` handles JWT tokens and auto-refresh automatically

9. Backend Guide

The backend is a **Node.js Express** REST API written in **TypeScript**, compiled with `tsx` in development and `tsc` for production Docker builds.

API Route Map

PREFIX	FILE	WHAT IT DOES
`POST /auth/login`	routes/auth.ts	Login, returns JWT access + refresh tokens
`POST /auth/refresh`	routes/auth.ts	Refresh access token using refresh cookie
`GET /mail/messages`	routes/mail.ts	List emails in a folder via IMAP
`POST /mail/send`	routes/mail.ts	Send email via SMTP
`GET /files/list`	routes/files.ts	Browse user files
`POST /files/upload`	routes/files.ts	Upload a file
`POST /files/share`	routes/files.ts	Share file/folder with another user
`GET /files/shared-with-me`	routes/files.ts	List files shared to current user
`GET /calendar/events`	routes/calendar.ts	Fetch CalDAV events
`GET /contacts`	routes/contacts.ts	Fetch CardDAV contacts
`GET /settings/profile`	routes/settings.ts	Get user profile (name, avatar)
`POST /wopi/token`	routes/wopi.ts	Get WOPI token for Collabora office editing
`GET /admin/users`	routes/adminPanel.ts	List users (admin only)
`GET /superadmin/tenants`	routes/tenants.ts	List tenants (superadmin only)

Authentication Flow

1. Client sends `POST /auth/login` with email + password
2. Backend verifies against Dovecot (IMAP login attempt) or MongoDB for admin/superadmin
3. Returns short-lived access token (15 min) in response + long-lived refresh token in httpOnly cookie
4. Client stores access token in Zustand (memory only — not localStorage)
5. On 401, the axios interceptor in `api/client.ts` auto-calls `POST /auth/refresh`

Adding a New API Route

1. Create `backend/src/routes/myRoute.ts`
2. Import and register in `backend/src/index.ts`: `app.use('/my-route', myRouter)`
3. Protect with JWT middleware: `router.use(requireAuth)` from `middleware/auth.ts`
4. Add Zod schema validation for request body

MongoDB Models

MODEL	FILE	PURPOSE

User	models/User.ts	User accounts (admin/superadmin — mail users are in Dovecot)
Tenant	models/Tenant.ts	Company tenants (superadmin managed)
Share	models/Share.ts	File sharing records (ownerEmail, path, sharedWith, permission)
Bill	models/Bill.ts	Billing records per tenant
SharedEvent	models/SharedEvent.ts	Shared calendar events

10. Database Guide

MongoDB

Used for: User accounts, Tenants, File shares, Billing. Mail data (actual emails) is stored by Dovecot in `D:\maildata\mail\` in Maildir format.

```
# Connect to MongoDB in Docker
docker exec -it mailserver-mongodb mongosh \
  -u mailserver -p changeme \
  --authenticationDatabase mailserver

# Common queries
use mailserver
db.users.find()           # List all admin/superadmin accounts
db.tenants.find()         # List all company tenants
db.shares.find()          # List all file shares
```

> ⚠️ In development with `bash dev.sh`, the backend connects to a LOCAL MongoDB instance (no auth) at `localhost:27017`. In Docker, it connects to `mongodb:27017` with auth credentials from `.env`.

Redis

Used for: JWT refresh token allowlist / blacklist (session invalidation on logout).

```
# Connect to Redis in Docker
docker exec -it mailserver-redis redis-cli -a $REDIS_PASSWORD
```

11. Docker Reference

Container Overview

CONTAINER	IMAGE	PURPOSE	PORT
mailserver-api	mailserver-api:local	Node.js REST API	3001 (host) → 3000 (internal)
mailserver-webmail	mailserver-webmail:local	React frontend (nginx)	internal only (via nginx proxy)
mailserver-nginx	nginx:alpine	Reverse proxy	8080, 8443
mailserver-mongodb	mongo	Database	27017 (internal)
mailserver-redis	redis:alpine	Token store	6379 (internal)
mailserver-dovecot	custom	IMAP server	993 (internal)
mailserver-postfix	custom	SMTP server	587 (internal)
mailserver-cloudflared	cloudflare/cloudflared	HTTPS tunnel	—
mailserver-collabora	collabora/code	Office editor	9980 (internal), 8082 (host dev)

Docker Commands Cheat Sheet

```
# View all running containers
docker ps

# View logs
docker logs mailserver-api -f
docker logs mailserver-webmail -f

# Restart a single container
docker compose -f docker-compose.apps.yml restart api

# Rebuild one container after code change
docker compose -f docker-compose.apps.yml build webmail
docker compose -f docker-compose.apps.yml up -d webmail

# Rebuild BOTH after changes to frontend AND backend
docker compose -f docker-compose.apps.yml build webmail api
docker compose -f docker-compose.apps.yml up -d webmail api

# Full teardown and rebuild
docker compose -f docker-compose.apps.yml down
docker compose -f docker-compose.apps.yml up -d --build

# Enter a running container shell
docker exec -it mailserver-api sh
docker exec -it mailserver-webmail sh
```

12. Making & Deploying Changes

Workflow for Frontend Changes

1. Edit files in `frontend/src/`
2. Test locally: `bash dev.sh` → open `http://localhost:5173`
3. When satisfied, rebuild Docker:

```
docker compose -f docker-compose.apps.yml build webmail
docker compose -f docker-compose.apps.yml up -d webmail
```

4. Do a hard refresh in browser: **Ctrl+Shift+R**

Workflow for Backend Changes

1. Edit files in `backend/src/`
2. Test locally: `bash dev.sh` — backend restarts automatically via `tsx watch`
3. When satisfied, rebuild:

```
docker compose -f docker-compose.apps.yml build api
docker compose -f docker-compose.apps.yml up -d api
```

Adding a New Feature — Checklist

STEP	FRONTEND	BACKEND
1	Create page/component in `frontend/src/pages/` or `components/`	Create route in `backend/src/routes/`
2	Add route in `App.tsx`	Register route in `backend/src/index.ts`
3	Add API function in `frontend/src/api/`	Add Mongoose model if needed in `models/`
4	Add nav link in `AppNavbar.tsx` and `Sidebar.tsx`	Add auth middleware `requireAuth` to route
5	Rebuild webmail container	Rebuild api container

13. User Roles & Login

All roles use the **same login page** at `/login`. After login, the server returns the user's role and the frontend redirects accordingly.

ROLE	STORED IN	REDIRECTED TO	CAN ACCESS
user	Dovecot (IMAP auth)	`/mail/INBOX`	Mail, Calendar, Files, Contacts, Settings
admin	MongoDB `users` collection	`/admin/users`	User management + all user features
superadmin	MongoDB `users` collection	`/superadmin/tenants`	Tenant & billing management

Creating a Superadmin Account

```
# Connect to MongoDB
docker exec -it mailserver-mongodb mongosh \
  -u mailserver -p changeme \
  --authenticationDatabase mailserver

# In mongosh — insert superadmin (password is hashed with argon2)
# Use the /setup endpoint instead:
curl -X POST http://localhost:3001/setup/init \
  -H "Content-Type: application/json" \
  -d '{"email":"admin@yourdomain.com","password":"yourpassword","role":"superadmin"}'
```

Login Page Location

The universal login page is at: `frontend/src/pages/Login.tsx`
Role-based redirect logic is in the `roleHome()` function at the top of that file.

14. File Storage

Files are stored on the Windows host at `D:\maildata\files\` and mounted into the `api` Docker container at `/data/files`.

Directory Structure

```
D:\maildata\files\
  ├── user-at-domain-com\      ← sanitized email (@ → -at-, . → -)
  │   ├── documents\
  │   ├── Sent\               ← attachments from sent emails
  │   └── received\
  │       └── sender-at-domain-com\  ← attachments saved from received emails
  └── anotheruserat-domain-com\
```

File Sharing

Sharing is implemented in `backend/src/routes/files.ts` using the `Share` Mongoose model. The share record stores: `ownerEmail`, `path`, `sharedWith` (array of emails), `permission` (view/edit).

Shared files appear in the "Shared with me" tab in the Files page.

15. Office Editing (WOPI / Collabora)

Clicking "Edit in Office" on a `.docx`, `.xlsx`, or `.pptx` file opens Collabora CODE editor in the browser. This uses the **WOPI protocol**.

Flow

```
1. Frontend calls POST /wopi/token  →  gets {token, wopiSrc, editorUrl}
2. Frontend opens Collabora at {editorUrl}?WOPISrc={wopiSrc}&access_token={token}
3. Collabora fetches file from  GET  /wopi/files/{fileId}     (our API)
4. Collabora saves file back to PUT  /wopi/files/{fileId}     (our API)
5. Our API reads/writes from D:\maildata\files\{user}\{path}
```

Config Variables

VARIABLE	DOCKER VALUE	DEV VALUE
`COLLABORA_URL`	`http://collabora:9980`	`http://localhost:8082`
`WOPI_HOST`	`http://api:3000`	`http://host.docker.internal:3000`

> In dev mode, Collabora (running in Docker) needs to reach the backend on the host machine. It uses `host.docker.internal` (Windows Docker Desktop resolves this to the host IP automatically).

16. Common Errors & Fixes

ERROR	CAUSE	FIX
Changes not visible after editing code	Accessing port 80 (Docker container) instead of port 5173 (dev server)	Use `http://localhost:5173` in dev. OR rebuild Docker container and hard refresh (Ctrl+Shift+R)
`Cannot find module 'tsx'`	Corrupt or missing node_modules in backend	`cd backend && rm -rf node_modules package-lock.json && npm install`
Port 5173 only accessible on IPv6	Vite not binding to IPv4	Ensure `vite.config.ts` has `server: { host: "0.0.0.0" }`
"Could not open editor" in Collabora	WOPI route not registered, wrong host config, or .env not loaded	Check `COLLABORA_URL` and `WOPI_HOST` env vars. Ensure `backend/.env` exists and has correct values. Restart backend.
MongoDB auth failing in dev	Local MongoDB has no users; Docker MongoDB requires credentials	Dev `backend/.env` should use `mongodb://localhost:27017/mailserver` (no auth)
Avatar/profile not loading	Token refresh wiping avatar from store	Fixed: `api/client.ts` refresh call preserves `cur.avatar`
Avatar color (Tailwind class missing)	Tailwind JIT purges dynamic class names	Use `style={{ backgroundColor: hexColor }}` instead of dynamic Tailwind classes
Rollup missing (Windows install)	Corrupt package-lock with wrong binary	`cd frontend && rm -rf node_modules package-lock.json && npm install`
Docker build fails on Windows paths	Windows backslash in docker-compose volume path	Use forward slashes: `D:/maildata/files:/data/files`

17. Key Files Quick Reference

WHAT YOU WANT TO CHANGE	FILE TO EDIT
Add/remove a page route	`frontend/src/App.tsx`
Top navbar (authenticated app)	`frontend/src/components/Layout/AppNavbar.tsx`
Left sidebar / mail folders	`frontend/src/components/Layout/Sidebar.tsx`
Landing page	`frontend/src/pages/LandingPage.tsx`
Login page	`frontend/src/pages/Login.tsx`
Colour themes	`frontend/src/lib/themes.ts`
Auth state (token, role, avatar)	`frontend/src/store/index.ts`
HTTP client (axios + JWT refresh)	`frontend/src/api/client.ts`
Mail API calls (IMAP)	`frontend/src/api/mailApi.ts`
Backend entry point	`backend/src/index.ts`
Environment variables config	`backend/src/config/index.ts`
Authentication (login / refresh)	`backend/src/routes/auth.ts`
Email send/receive	`backend/src/routes/mail.ts`
File operations + sharing	`backend/src/routes/files.ts`
WOPI / office editing	`backend/src/routes/wopi.ts`
Nginx config (routing rules)	`config/nginx/conf.d/`
Docker services (app layer)	`docker-compose.apps.yml`
Docker services (mail core)	`docker-compose.yml`
Secrets / env vars	`.env` (root) and `backend/.env`

18. Checklist for New Developer

> Complete these steps before starting any development work.

☐	TASK	NOTES
☐	Install Node.js v20 LTS	Download from nodejs.org
☐	Install Docker Desktop	Enable WSL2 backend on Windows
☐	Clone the repo & check out `main` branch	Git on Windows
☐	Create `D:\maildata\files\` directory	Or update path in `.env`
☐	Copy `.env.example` to `.env` and fill secrets	Root directory
☐	Copy `backend/.env.example` to `backend/.env`	Set dev MongoDB URI
☐	Run `bash dev.sh --install`	Installs npm deps + starts dev servers
☐	Open `http://localhost:5173` and verify landing page loads	—
☐	Start Docker core services: `docker compose -f docker-compose.yml up -d`	MongoDB, Redis, Dovecot, Postfix
☐	Try logging in at `http://localhost:5173/login`	Create user via admin panel first
☐	Read `DEV.md` and `README.md` in the project root	Additional context
☐	Understand the Docker two-compose pattern	`docker-compose.yml` = mail core, `docker-compose.apps.yml` = app layer

Important Reminders

- Never commit `.env` files with real secrets to Git
- Always use `http://localhost:5173` in dev, not port 80
- After any code change → rebuild the Docker container before testing in production
- Browser hard refresh (**Ctrl+Shift+R**) after container rebuild
- The `D:\maildata\` folder contains all user data — back it up regularly
- MongoDB user data and mail accounts are separate: mail users live in Dovecot, admin/superadmin live in MongoDB